



INSTITUTO POLITECNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO



Practica 4:

APLICACIÓN DE LA TÉCNICA DE VALIDACIÓN CRUZADA EN PYTHON

Inteligencia Artificial

Profra. Vanessa Alejandra Camacho Vázquez

Grupo: 6CM3

Nombres de los integrantes:

- Aguirre Lanto Víctor Manuel
- Gasca Frago Pedro
- Guevara Badillo Areli Alejandra
- Montiel Toro Arael de Jesús
- Ramírez Lozano Gael Martin

Notebook: [Practica no.4](#)

viernes, 24 de octubre de 2025

CONTENIDO

MARCO TEÓRICO	3
PROCEDIMIENTO	7
DATASET IRIS().....	7
DATASET VINOS()	10
DIFERENCIAS ENTRE VALIDACIÓN CRUZADA Y 80%-20%	13
<i>¿Cuándo usar cada uno?</i>	13
CONCLUSIONES	14
AGUIRRE LANTO VÍCTOR MANUEL.....	14
GASCA FRAGOSO PEDRO	14
GUEVARA BADILLO ARELI ALEJANDRA	14
MONTIEL TORO ARAEL DE JESÚS.....	15
RAMÍREZ LOZANO GAELE MARTIN.....	15



MARCO TEÓRICO

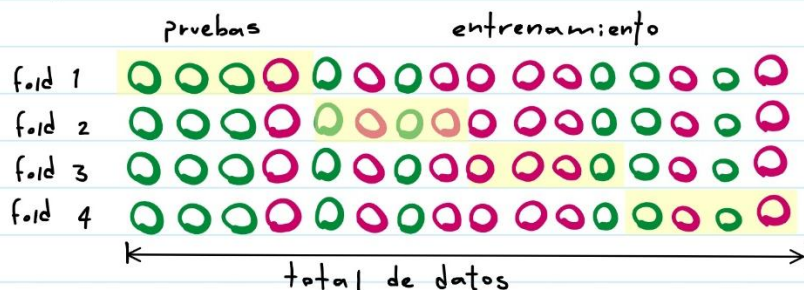
Técnica de validación cruzada

La validación cruzada es una técnica estadística fundamental en el aprendizaje automático diseñada para evaluar de manera robusta la capacidad de un modelo para generalizar a datos nuevos e independientes. Su objetivo principal es proporcionar una estimación más fiable y estable del rendimiento del modelo en el mundo real, mitigando problemas comunes como el sobreajuste (overfitting), que ocurre cuando un modelo aprende el ruido de los datos de entrenamiento en lugar de los patrones subyacentes.

A diferencia del método simple de división en entrenamiento y prueba (train-test split), que puede ser muy sensible a cómo se realiza la partición aleatoria de los datos, la validación cruzada utiliza múltiples divisiones para obtener una evaluación más completa. Al promediar el rendimiento a través de varias iteraciones, se reduce el sesgo y la varianza en la estimación final del rendimiento del modelo.

La implementación más popular de esta técnica es la validación cruzada de K pliegues (K-Fold Cross-validation). Este método equilibra la robustez de la evaluación con el costo computacional y funciona de la siguiente manera:

1. **Mezcla y División:** el conjunto de datos completo se mezcla de forma aleatoria. Luego, se divide en K subconjuntos (folds) de tamaño aproximadamente igual de tamaño K , comúnmente 5 o 10.
2. **Ciclo Iterativo:** El proceso se repite K veces. En cada iteración, un pliegue diferente se utiliza como conjunto de prueba, y los pliegues restantes se combinan para formar el conjunto de entrenamiento.
3. **Entrenamiento y Evaluación:** El modelo se entrena con el conjunto de entrenamiento y luego se evalúa con el pliegue de validación que se mantuvo al margen. La métrica de rendimiento de esta iteración se almacena.
4. **Agregación de Resultados:** Después de completar las K iteraciones, cada pliegue ha servido exactamente una vez como conjunto de validación. El rendimiento final del modelo se calcula promediando las K métricas de rendimiento obtenidas en cada iteración.



ejemplo:

Imaginemos un conjunto de datos ficticio de 12 frutas. Queremos clasificar cada una como Manzana (🍏) o Naranja (🍊) basándonos en su Rugosidad (escala 1-10).

id	rugosidad	clase
1	2	🍏
2	7	🍊
3	3	🍏
4	8	🍊
5	1	🍏
6	9	🍊
7	6	🍊
8	2	🍏
9	4	🍏
10	9	🍊
11	1	🍏
12	7	🍊

Paso 1: Dividir en 3 Pliegos (K=3)

Después de mezclar los datos, los dividimos en 3 pliegos de 4 frutas cada uno:

{ID 5, ID 12, ID 3, ID 7} {ID 1, ID 6, ID 11, ID 4} {ID 9, ID 2, ID 8, ID 10}

Paso 2: Ciclo Iterativo

Usaremos un modelo simple: "Si Rugosidad > 5, es 🍊; si no, es 🍏".

Iteración 1:

Entrenamiento: Pliegos 2 y 3. validación: Pliegue 1.

Resultado: El modelo clasifica correctamente las 4 frutas del Pliegue 1.

Precisión Iteración 1: 100%.

Iteración 2:

Entrenamiento: Pliegos 1 y 3. validación: Pliegue 2.

Resultado: El modelo clasifica correctamente las 4 frutas del Pliegue 2.

Precisión Iteración 2: 100%.

Iteración 3:

Entrenamiento: Pliegos 1 y 2. validación: Pliegue 3.

Resultado: El modelo clasifica correctamente las 4 frutas del Pliegue 3.

Precisión Iteración 3: 100%.

Paso 3: Resultado Final

Las puntuaciones de precisión de las tres iteraciones son [100%, 100%, 100%].

La precisión final de la validación cruzada es el promedio de estas puntuaciones:

$$Precision_{cv} = \frac{100\% + 100\% + 100\%}{3} = 100\%$$

Este resultado del 100% es una estimación mucho más confiable que si hubiéramos obtenido el mismo valor con una única división de los datos.

cross_validate()

La función `cross_validate()` es una herramienta de validación cruzada que ofrece mayor flexibilidad y devuelve más información que `cross_val_score()`.

Sus principales ventajas son:

1. **Evaluación de Múltiples Métricas:** Permite especificar y calcular varias métricas de rendimiento simultáneamente en una sola ejecución.
2. **Información Adicional:** Devuelve un diccionario que no solo contiene las puntuaciones de prueba (`test_score`), sino también información adicional como los tiempos de entrenamiento (`fit_time`), los tiempos de evaluación (`score_time`) y, opcionalmente, las puntuaciones del conjunto de entrenamiento (`train_score`).

Esta información adicional es muy útil para analizar el comportamiento del modelo, como el costo computacional y el grado de sobreajuste.

Descripción de Parámetros

`cv`: determina la estrategia de división para la validación cruzada. Puede aceptar varios tipos de entrada:

Un número entero (`int`): Especifica el número de pliegues a utilizar.

Un objeto divisor de CV: Se puede pasar una instancia de un divisor de validación cruzada, como `KFold` o `StratifiedKFold`, para tener un control más granular sobre el proceso de división.

Un iterable: Puede generar manualmente las divisiones de entrenamiento/prueba como una secuencia de tuplas de índices.

`scoring`: define la métrica o métricas que se utilizarán para evaluar el modelo. Ofrece varias opciones:

Una cadena de texto (`string`): El nombre de una única métrica predefinida en Scikit-learn, como `'accuracy'`, `'precision'`, `'recall'`, `'f1'` o `'roc_auc'` para clasificación, y `'r2'` o `'neg_mean_squared_error'` para regresión.

Una lista o tupla de cadenas: Para evaluar múltiples métricas a la vez.

Un diccionario (`dict`): Permite asignar nombres personalizados a las métricas.

`return_train_score`: un parámetro booleano (`True` o `False`) que, por defecto, es `False`.

Si se establece en `True`, el diccionario de resultados devuelto por `cross_validate()` incluirá una clave adicional, `train_score` (o `train_(nombre_metrica)` si se usan múltiples métricas). Esta clave contendrá las puntuaciones del modelo en los datos de entrenamiento para cada pliegue.

Ejemplo de uso

```

1 import numpy as np
2 from sklearn.datasets import make_classification
3 from sklearn.model_selection import cross_validate
4 from sklearn.ensemble import RandomForestClassifier
5
6 # 1. Crear un conjunto de datos de ejemplo
7 X, y = make_classification(n_samples=100, n_features=20, n_informative=2, n_redundant=10, random_state=42)
8
9 # 2. Inicializar el modelo
10 modelo = RandomForestClassifier(random_state=42)
11
12 # 3. Definir las métricas de evaluación
13 metricas = ['accuracy', 'precision_macro', 'recall_macro', 'f1_macro']
14
15 # 4. Ejecutar cross_validate()
16 # - cv=5: Validación cruzada de 5 pliegues.
17 # - scoring=metricas: Evalúa las cuatro métricas definidas.
18 # - return_train_score=True: Incluye las puntuaciones del conjunto de entrenamiento.
19 resultados_cv = cross_validate(modelo, X, y, cv=5, scoring=metricas, return_train_score=True)
20
21 # 5. Imprimir los resultados
22 print("Resultados de la Validación Cruzada:")
23 print("-" * 35)
24
25 # Puntuaciones promedio en los pliegues de prueba
26 print(f"Precisión (Accuracy) en prueba: {np.mean(resultados_cv['test_accuracy']):.3f}")
27 print(f"Precisión (Precision) en prueba: {np.mean(resultados_cv['test_precision_macro']):.3f}")
28 print(f"Sensibilidad (Recall) en prueba: {np.mean(resultados_cv['test_recall_macro']):.3f}")
29 print(f"Puntuación F1 en prueba: {np.mean(resultados_cv['test_f1_macro']):.3f}")
30 print("-" * 35)
31
32 # Puntuaciones promedio en los pliegues de entrenamiento para verificar sobreajuste
33 print(f"Precisión (Accuracy) en entrenamiento: {np.mean(resultados_cv['train_accuracy']):.3f}")
34 print(f"Puntuación F1 en entrenamiento: {np.mean(resultados_cv['train_f1_macro']):.3f}")
35 print("-" * 35)
36
37 # Tiempos de ejecución
38 print(f"Tiempo promedio de ajuste (fit): {np.mean(resultados_cv['fit_time']):.3f} segundos")
39 print(f"Tiempo promedio de puntuación (score): {np.mean(resultados_cv['score_time']):.3f} segundos")

```

Resultados de la Validación Cruzada:

```

-----
Precisión (Accuracy) en prueba: 0.970
Precisión (Precision) en prueba: 0.974
Sensibilidad (Recall) en prueba: 0.970
Puntuación F1 en prueba: 0.970

```

```

-----
Precisión (Accuracy) en entrenamiento: 1.000
Puntuación F1 en entrenamiento: 1.000

```

```

-----
Tiempo promedio de ajuste (fit): 0.409 segundos
Tiempo promedio de puntuación (score): 0.044 segundos

```


PROCEDIMIENTO

Dataset Iris()

Este código usa validación cruzada con 5 iteraciones en un clasificador basado en el método Naive Bayes, y luego muestra los promedios de las métricas de precisión, exactitud, recall y F-score.

```
from sklearn.model_selection import cross_validate
from sklearn.naive_bayes import GaussianNB
from sklearn import datasets

# Cargar un dataset de ejemplo
iris = datasets.load_iris()
X = iris.data
y = iris.target

# Crear el clasificador Naive Bayes
clf = GaussianNB()

# Definir las métricas que deseas calcular
scoring = ['accuracy', 'precision_macro', 'recall_macro', 'f1_macro']

# Aplicar validación cruzada con múltiples métricas
scores = cross_validate(clf, X, y, cv=5, scoring=scoring)

# Imprimir los resultados de las métricas
print("Accuracy:", scores['test_accuracy'].mean())
print("Precision:", scores['test_precision_macro'].mean())
print("Recall:", scores['test_recall_macro'].mean())
print("F-score:", scores['test_f1_macro'].mean())
```

```
Accuracy: 0.9533333333333334
Precision: 0.9583838383838383
Recall: 0.9533333333333331
F-score: 0.9530472646262119
```

Al imprimir la variable scores se observará una matriz de puntajes, donde cada elemento corresponde al rendimiento del modelo en una iteración de la validación cruzada.

```
import numpy as np
import pandas as pd
from IPython.display import display

# Convertir el diccionario 'scores' en un DataFrame para que se vea mas bonito :D
df = pd.DataFrame({k: np.asarray(v) for k, v in scores.items()})

print("\n--- Matriz de Puntajes (scores) ---")
display(df.round(6))
```

```
--- Matriz de Puntajes (scores) ---
```

	fit_time	score_time	test_accuracy	test_precision_macro	test_recall_macro	test_f1_macro
0	0.002776	0.019623	0.933333	0.933333	0.933333	0.933333
1	0.002129	0.011219	0.966667	0.969697	0.966667	0.966583
2	0.001677	0.009804	0.933333	0.944444	0.933333	0.932660
3	0.001809	0.009616	0.933333	0.944444	0.933333	0.932660
4	0.001647	0.009305	1.000000	1.000000	1.000000	1.000000

Estos puntajes pueden ser utilizados para evaluar el rendimiento promedio del modelo, calcular intervalos de confianza, comparar diferentes modelos, entre otros usos.

Comparación visual de cada iteración

```

# Extraer los datos de las métricas para la gráfica
metrics = ['test_accuracy', 'test_precision_macro', 'test_recall_macro', 'test_f1_macro']
results = {m: scores[m] for m in metrics}

# Nombres más amigables para el eje Y
metric_names = ['Accuracy', 'Precision', 'Recall', 'F1-Score']

# Preparar la gráfica
n_folds = len(scores['test_accuracy']) # Número de iteraciones (cv=5)
x = np.arange(n_folds) # Posiciones para las barras
width = 0.2 # Ancho de las barras

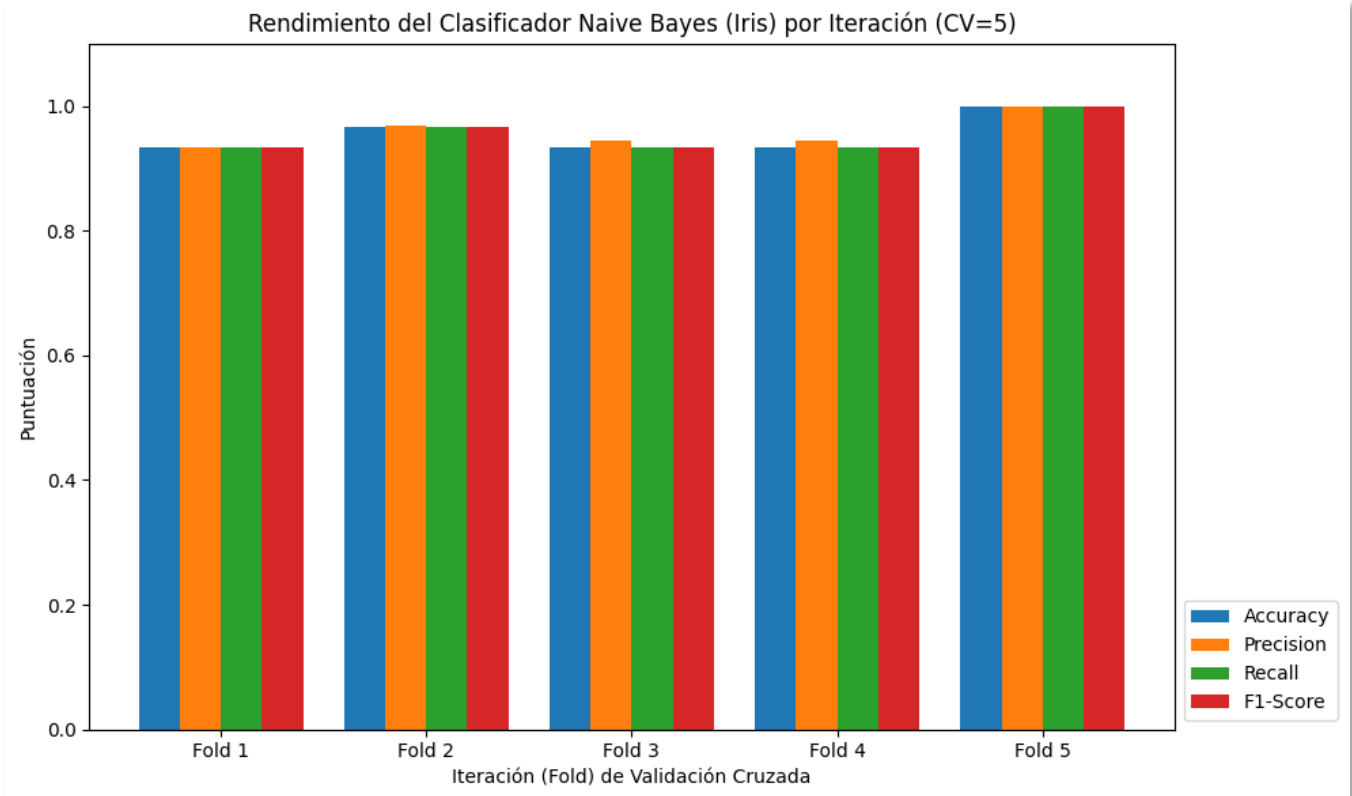
fig, ax = plt.subplots(figsize=(10, 6))

# Dibujar las barras para cada métrica
rects1 = ax.bar(x - 1.5*width, results['test_accuracy'], width, label=metric_names[0])
rects2 = ax.bar(x - 0.5*width, results['test_precision_macro'], width, label=metric_names[1])
rects3 = ax.bar(x + 0.5*width, results['test_recall_macro'], width, label=metric_names[2])
rects4 = ax.bar(x + 1.5*width, results['test_f1_macro'], width, label=metric_names[3])

# Añadir etiquetas, título y leyenda
ax.set_ylabel('Puntuación')
ax.set_xlabel('Iteración (Fold) de Validación Cruzada')
ax.set_title('Rendimiento del Clasificador Naive Bayes (Iris) por Iteración (CV=5)')
ax.set_xticks(x)
ax.set_xticklabels([f'Fold {i+1}' for i in range(n_folds)])
ax.legend(loc='lower left', bbox_to_anchor=(1, 0))
ax.set_ylim(0, 1.1) # Las puntuaciones van de 0 a 1

plt.tight_layout()
plt.show() # Mostrar la gráfica

```



Utilizando el mismo conjunto de datos de entrada y el mismo clasificador de Naive Bayes pero sin aplicar la técnica de validación cruzada – solamente usando el 80% de las muestras para entrenamiento y el 20% para pruebas, obtuvimos los siguientes resultados de las medidas de accuracy, precisión, recall y F-score.

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Separar en 80% entrenamiento y 20% prueba
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

# Crear y entrenar el clasificador Naive Bayes
clf = GaussianNB()
clf.fit(X_train, y_train)

# Predecir sobre el conjunto de prueba
y_pred = clf.predict(X_test)

# Calcular métricas (macro para que trate por igual cada clase)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='macro', zero_division=0)
recall = recall_score(y_test, y_pred, average='macro', zero_division=0)
f1 = f1_score(y_test, y_pred, average='macro', zero_division=0)

# Imprimir resultados
print("Resultados (80% train / 20% test):")
print(f"Número de muestras de entrenamiento (80%): {len(X_train)}")
print(f"Número de muestras de prueba (20%): {len(X_test)}")
print("-" * 45)
print(f"Accuracy:  {accuracy:.4f}")
print(f"Precision: {precision:.4f} (macro)")
print(f"Recall:    {recall:.4f} (macro)")
print(f"F1-score:   {f1:.4f} (macro)\n")
```

```
Resultados (80% train / 20% test):
Número de muestras de entrenamiento (80%): 120
Número de muestras de prueba (20%): 30
-----
Accuracy:  0.9667
Precision: 0.9697 (macro)
Recall:    0.9667 (macro)
F1-score:  0.9666 (macro)
```

Dataset Vinos()

```
from sklearn.model_selection import cross_validate
from sklearn.naive_bayes import GaussianNB
from sklearn import datasets

# Cargar el dataset de vinos (wine)
wine = datasets.load_wine()
X = wine.data
y = wine.target

# Crear el clasificador Naive Bayes
clf2 = GaussianNB()

# Definir las métricas que deseas calcular
scoring2 = ['accuracy', 'precision_macro', 'recall_macro', 'f1_macro']

# Aplicar validación cruzada con múltiples métricas
scores2 = cross_validate(clf2, X, y, cv=5, scoring=scoring2)

# Imprimir los resultados de las métricas (promedio de CV)
print("--- Resultados de Validación Cruzada (CV=5) (Promedios) ---")
print(f"Accuracy: {scores2['test_accuracy'].mean():.4f}")
print(f"Precision: {scores2['test_precision_macro'].mean():.4f}")
print(f"Recall: {scores2['test_recall_macro'].mean():.4f}")
print(f"F-score: {scores2['test_f1_macro'].mean():.4f}")

--- Resultados de Validación Cruzada (CV=5) (Promedios) ---
Accuracy: 0.9663
Precision: 0.9700
Recall: 0.9690
F-score: 0.9677
```

```
import numpy as np
import matplotlib.pyplot as plt

# Convertir el diccionario 'scores' en un DataFrame para que se vea mas bonito :D
df = pd.DataFrame({k: np.asarray(v) for k, v in scores2.items()})

print("\n--- Matriz de Puntuajes (scores) ---")
display(df.round(6))
```

```
--- Matriz de Puntuajes (scores) ---
```

	fit_time	score_time	test_accuracy	test_precision_macro	test_recall_macro	test_f1_macro
0	0.002044	0.011311	0.944444	0.944056	0.952381	0.945153
1	0.001895	0.012130	0.972222	0.969697	0.976190	0.971781
2	0.001882	0.019732	0.972222	0.977778	0.972222	0.974013
3	0.004166	0.023148	0.942857	0.958333	0.944444	0.947475
4	0.004313	0.014449	1.000000	1.000000	1.000000	1.000000

Comparación visual de cada iteración

```
# Extraer los datos de las métricas para la gráfica
metrics2 = ['test_accuracy', 'test_precision_macro', 'test_recall_macro', 'test_f1_macro']
results2 = {m: scores2[m] for m in metrics2}

# Nombres más amigables para el eje Y
metric_names2 = ['Accuracy', 'Precision', 'Recall', 'F1-Score']

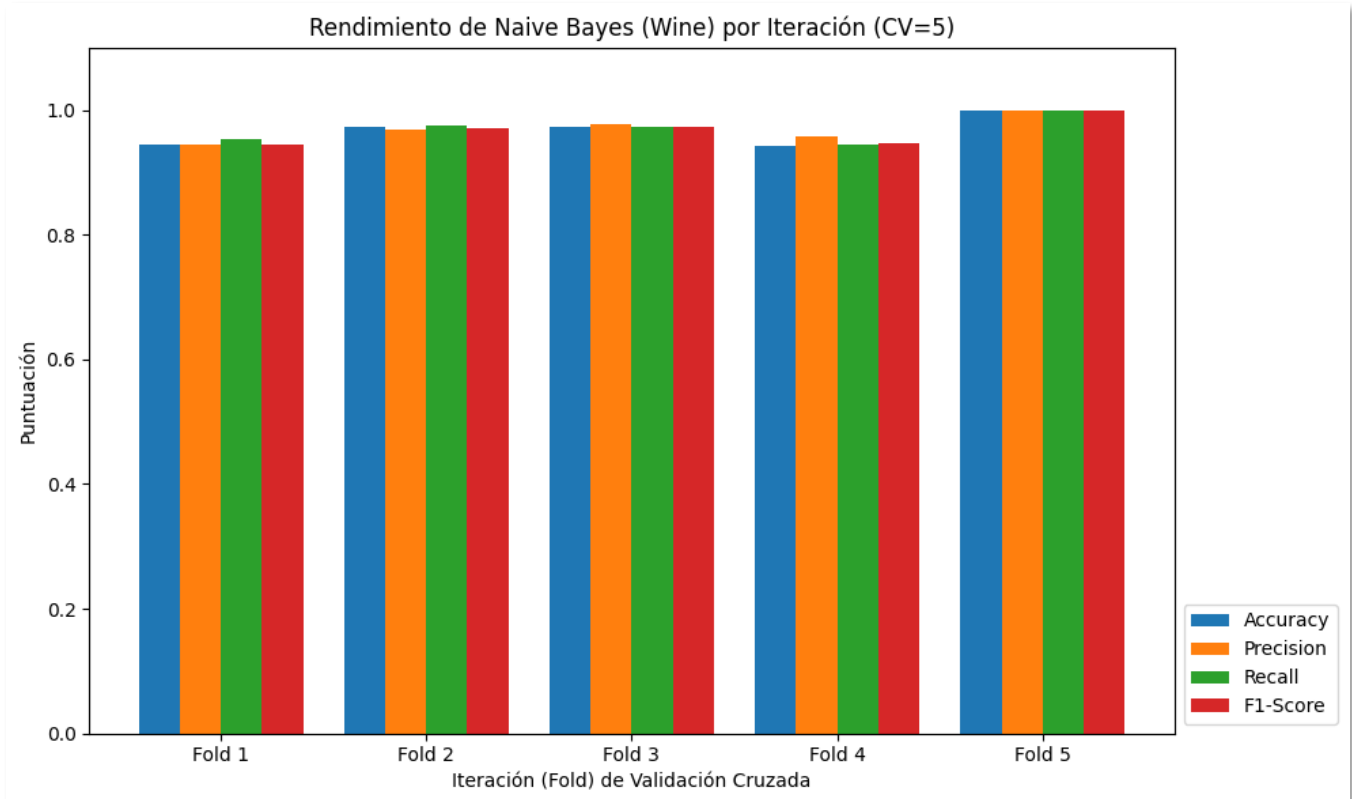
# Preparar la gráfica
n_folds = len(scores2['test_accuracy']) # Número de iteraciones (cv=5)
x = np.arange(n_folds) # Posiciones para las barras
width = 0.2 # Ancho de las barras

fig, ax = plt.subplots(figsize=(10, 6))

# Dibujar las barras para cada métrica
rects1 = ax.bar(x - 1.5*width, results2['test_accuracy'], width, label=metric_names[0])
rects2 = ax.bar(x - 0.5*width, results2['test_precision_macro'], width, label=metric_names[1])
rects3 = ax.bar(x + 0.5*width, results2['test_recall_macro'], width, label=metric_names[2])
rects4 = ax.bar(x + 1.5*width, results2['test_f1_macro'], width, label=metric_names[3])

# Añadir etiquetas, título y leyenda
ax.set_ylabel('Puntuación')
ax.set_xlabel('Iteración (Fold) de Validación Cruzada')
ax.set_title('Rendimiento de Naive Bayes (Wine) por Iteración (CV=5)')
ax.set_xticks(x)
ax.set_xticklabels([f'Fold {i+1}' for i in range(n_folds)])
ax.legend(loc='lower left', bbox_to_anchor=(1, 0))
ax.set_ylim(0, 1.1) # Las puntuaciones van de 0 a 1

plt.tight_layout()
plt.show()
```



Utilizando el mismo conjunto de datos de entrada y el mismo clasificador de Naive Bayes pero sin aplicar la técnica de validación cruzada – solamente usando el 80% de las muestras para entrenamiento y el 20% para pruebas, obtuvimos los siguientes resultados de las medidas de accuracy, precisión, recall y F-score.

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Separar en 80% entrenamiento y 20% prueba
# Usamos las variables X e y del dataset Wine (cargadas en Parte 1)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

# Crear y entrenar el clasificador Naive Bayes
clf_split = GaussianNB() # Usamos una nueva instancia para claridad
clf_split.fit(X_train, y_train)

# Predecir sobre el conjunto de prueba
y_pred = clf_split.predict(X_test)

# Calcular métricas (macro para que trate por igual cada clase)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='macro', zero_division=0)
recall = recall_score(y_test, y_pred, average='macro', zero_division=0)
f1 = f1_score(y_test, y_pred, average='macro', zero_division=0)

# Imprimir resultados
print("\n--- Resultados (80% train / 20% test) ---")
print(f"Número de muestras de entrenamiento (80%): {len(X_train)}")
print(f"Número de muestras de prueba (20%): {len(X_test)}")
print("-" * 45)
print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f} (macro)")
print(f"Recall: {recall:.4f} (macro)")
print(f"F1-score: {f1:.4f} (macro)\n")
```

```
--- Resultados (80% train / 20% test) ---
Número de muestras de entrenamiento (80%): 142
Número de muestras de prueba (20%): 36
-----
Accuracy:  0.9722
Precision: 0.9744 (macro)
Recall:    0.9762 (macro)
F1-score:  0.9743 (macro)
```

Diferencias entre validación cruzada y 80%-20%

1. División de datos:

- La validación cruzada divide los datos en varios grupos (folds) y entrena el modelo varias veces.
- El método 80%-20% solo divide una vez: 80% para entrenar y 20% para probar.

2. Precisión de resultados:

- La validación cruzada da resultados más confiables porque usa distintas combinaciones de datos.
- El 80%-20% puede variar más dependiendo de cómo se haga la división.

3. Tiempo de ejecución:

- La validación cruzada tarda más porque repite el entrenamiento varias veces.
- El 80%-20% es más rápido y fácil de aplicar.

¿Cuándo usar cada uno?

Se usa **validación cruzada** cuando hay pocos datos o se busca una evaluación más precisa.

Se usa **80%-20%** cuando hay muchos datos o se necesita un resultado rápido.

CONCLUSIONES

Aguirre Lanto Víctor Manuel

Desde mi perspectiva, la validación cruzada representa una herramienta fundamental en el análisis y evaluación de modelos de aprendizaje automático, especialmente cuando se trabaja con conjuntos de datos limitados. Esta técnica permite aprovechar de manera más eficiente toda la información disponible, ya que cada muestra participa tanto en el entrenamiento como en la validación en diferentes momentos. Gracias a este proceso iterativo, se obtiene una medida más estable del rendimiento general del modelo, reduciendo la dependencia de una única partición de los datos. Aunque su ejecución puede demandar más tiempo y recursos computacionales, los resultados tienden a ser más confiables y consistentes, lo que la convierte en una opción ideal para experimentos donde la precisión es prioritaria.

Gasca Fragoso Pedro

Desde mi punto de vista, el método 80%-20% resulta muy práctico cuando se necesita rapidez en los experimentos o cuando se cuenta con una base de datos grande. Sin embargo, su principal limitación es que una sola división puede no reflejar adecuadamente la variabilidad de los datos, lo que podría afectar la fiabilidad del resultado final.

Guevara Badillo Areli Alejandra

Considero que la elección entre validación cruzada y el método 80%-20% depende directamente del propósito del análisis y los recursos disponibles. Si el interés radica en realizar una evaluación profunda y precisa del modelo, la validación cruzada ofrece una ventaja significativa al promediar los resultados obtenidos de diferentes divisiones de los datos.

Montiel Toro Arael de Jesús

Considero que la validación cruzada es una técnica más completa para evaluar el rendimiento de un modelo, ya que permite comprobar su estabilidad con diferentes subconjuntos de datos. A diferencia del segundo método, no depende de una única división, lo que ayuda a obtener resultados más justos y representativos del comportamiento real del modelo.

Ramírez Lozano Gael Martin

En mi opinión, el método de división 80%-20% es una alternativa válida y muy útil en escenarios donde se dispone de grandes volúmenes de datos o cuando el objetivo principal es obtener resultados rápidos. Este método es sencillo de aplicar, fácil de interpretar y permite evaluar el desempeño del modelo de manera inmediata sin necesidad de repetir múltiples entrenamientos.